

Procedimiento de analisis estatico profundo de muestras PowerPoint maliciosas

PPT, PPS, PPTX, PPSX, PPTM y PPSM

Guia operativa para analisis estatico, extraccion, correlacion y preparacion para
emulacion/dinamico

Objetivo. Este documento define un procedimiento reproducible para analizar muestras de malware distribuidas como presentaciones de Microsoft PowerPoint. El flujo se adapta al tipo de fichero: presentaciones antiguas en formato OLE/CFBF (.ppt/.pps) y presentaciones Office Open XML basadas en ZIP (.pptx/.ppsx/.pptm/.ppsm).

Extension	Formato base	Riesgo principal	Herramientas iniciales
.ppt / .pps	OLE/CFBF	Macros, streams OLE, objetos embebidos, PE/shellcode	oleid, oledump.py, olevba, oleobj
.pptx / .ppsx	OOXML/ZIP	Relaciones externas, objetos embebidos, ActiveX, enlaces	unzip/7z, grep, zipdump.py, oleobj
.pptm / .ppsm	OOXML/ZIP + VBA	Macros VBA, vbaProject.bin, objetos embebidos	olevba, oledump.py, oleid

Indice

- 0. Flujo general
- 1. Informacion general de la muestra
- 2. Identificacion del tipo de PowerPoint
- 3. Analisis estatico inicial
- 4. Deteccion de ofuscacion
- 5. Extraccion de objetos y artefactos
- 6. Analisis de artefactos extraidos
- 7. Busqueda generica de cadenas
- 8. Correlacion con la estructura original
- 9. Analisis de contexto hexadecimal y desensamblado
- 10. Clasificacion de hallazgos
- 11. Conclusion estatica prudente
- 12. Cuándo pasar a emulacion o analisis dinamico
- 13. Checklist rapido

0. Flujo general

identificacion -> tipo de PowerPoint -> estructura -> ofuscacion -> extraccion -> analisis de artefactos -> correlacion -> interpretacion tecnica -> emulacion/dinamico si procede

La idea principal es no tratar todas las presentaciones igual. Un fichero **.ppt** antiguo se analiza como contenedor OLE/CFBF, mientras que un **.pptx/.pptm** se analiza como un paquete ZIP/OOXML. El procedimiento general es el mismo, pero los artefactos, rutas internas y herramientas cambian.

1. Informacion general de la muestra

1.1 Identificacion del fichero

```
file muestra
```

```
exiftool muestra > exiftool.txt
```

- Confirmar si la extension coincide con el formato real.
- Diferenciar entre OLE/CFBF y OOXML/ZIP.
- Registrar tamaño, fechas y metadatos basicos.
- Detectar extensiones alteradas, por ejemplo un ZIP renombrado como .ppt.

1.2 Hashes y trazabilidad

```
md5sum muestra
```

```
sha1sum muestra
```

```
sha256sum muestra
```

Los hashes permiten identificar la muestra de forma inequívoca durante todo el análisis y compararla con informes externos o sandboxes.

1.3 Consultas externas

- VirusTotal, MalwareBazaar, Joe Sandbox, Hybrid Analysis u otros servicios pueden usarse como contexto.
- No se deben usar como sustituto del analisis propio.
- Si un sandbox informa de URLs, procesos o ficheros, tratarlos como hipotesis a confirmar.

2. Identificacion del tipo de PowerPoint

Antes de extraer nada es esencial clasificar el contenedor. El flujo cambia segun sea OLE/CFBF o OOXML/ZIP.

```
file muestra.ppt
file muestra.pptx
xxd -l 16 muestra
```

Firma/cabecera	Significado	Accion
D0 CF 11 E0 A1 B1 1A E1	OLE/CFBF	Analizar con oleid, oledump.py, olevba y oleobj
50 4B 03 04	ZIP/OOXML	Extraer con unzip/7z y revisar estructura interna
4D 5A	PE/EXE	Tratar como ejecutable; no abrir como documento

2.1 Formatos OLE: .ppt y .pps

- Son contenedores OLE Compound File Binary Format.
- Pueden contener streams, macros VBA, objetos embebidos y datos binarios.
- El analisis se centra en storages, streams, macros y posibles objetos OLE incrustados.

```
oleid muestra.ppt
oledump.py muestra.ppt
oledump.py --storages muestra.ppt
olevba muestra.ppt
```

2.2 Formatos OOXML: .pptx, .ppsx, .pptm y .ppsm

- Son paquetes ZIP con XML, relaciones y recursos internos.
- .pptm y .ppsm pueden contener macros en ppt/vbaProject.bin.
- Los objetos embebidos suelen estar en ppt/embeddings/.
- Las relaciones externas se documentan en ficheros .rels.

```
unzip -l muestra.pptx | head
mkdir pptx_extract
unzip muestra.pptx -d pptx_extract
find pptx_extract -type f | sort
```

3. Analisis estatico inicial

3.1 Metadatos con ExifTool

```
exiftool muestra > exiftool.txt
```

- Revisar tipo de fichero, fechas, aplicacion generadora y metadatos de autor.
- Buscar referencias a objetos embebidos, OLE, macros o rutas.
- En OOXML, los metadatos tambien pueden aparecer en docProps/core.xml y docProps/app.xml.

3.2 Analisis inicial de .ppt/.pps con oletools

```
oleid muestra.ppt
```

```
olevba muestra.ppt
```

```
mraptor muestra.ppt
```

```
oleobj muestra.ppt
```

- oleid ofrece una vista rapida de macros, OLE, cifrado y posibles indicadores.
- olevba extrae y analiza macros VBA si existen.
- mraptor puede ayudar a detectar patrones de macros maliciosas.
- oleobj intenta extraer objetos OLE embebidos.

3.3 Analisis inicial de .pptx/.pptm como OOXML

```
unzip -l muestra.pptx
```

```
find pptx_extract -type f | sort
```

```
find pptx_extract -type f -name "*.rels" -print
```

- Revisar [Content_Types].xml para conocer tipos de contenido declarados.
- Revisar ppt/_rels/presentation.xml.rels y ppt/slides/_rels/*.rels.
- Buscar TargetMode="External", enlaces remotos y referencias a objetos embebidos.
- Revisar ppt/embeddings/, ppt/activeX/, ppt/media/ y ppt/vbaProject.bin.

```
grep -Rai "TargetMode=\"External\"\\|http\\|https\\|file:\\|oleObject\\|activeX" pptx_extract
```

4. Deteccion de ofuscacion

La ofuscacion en PowerPoint puede aparecer en macros, XML, relationships, objetos OLE, ActiveX, scripts embebidos o blobs binarios. No todas las tecnicas aparecern en todas las muestras.

4.1 Búsqueda inicial de indicios

```
strings -a muestra | grep -Ei "http|https|cmd|powershell|wscript|mshta|exe|dll|CreateObject|Shell|XMLHTTP|ADODB|Download"
```

```
strings -a -el muestra | grep -Ei "http|https|cmd|powershell|wscript|mshta|exe|dll|CreateObject|Shell|XMLHTTP|ADODB|Download"
```

4.2 Ofuscacion en macros VBA

- Concatenacion de cadenas.
- Chr, ChrW, Asc, StrReverse, Replace, Mid, Left, Right.

Procedimiento de analisis estatico de PowerPoint maliciosos

- Base64, hexadecimal textual o arrays de bytes.
- CreateObject con WScript.Shell, MSXML2.XMLHTTP, WinHttpRequest o ADODB.Stream.
- AutoOpen, Auto_Open, Presentation_Open, SlideShowBegin u otros eventos.

```
olevba muestra.pptm > olevba.txt
```

```
grep -Ei "Auto|Open|CreateObject|Shell|PowerShell|XMLHTTP|ADODB|Base64|Chr|StrReverse" olevba.txt
```

4.3 Ofuscacion en OOXML y relationships

- Relaciones externas en ficheros .rels.
- TargetMode="External" apuntando a http, https, file, smb o rutas UNC.
- Objetos OLE en ppt/embeddings/ con extensiones genericas.
- ActiveX en ppt/activeX/ y referencias asociadas.
- URLs o rutas partidas entre varios XML.

```
grep -Rai "TargetMode=\"External\"" pptx_extract
```

```
grep -Rai "http\\|https\\|file:\\|\\\\" pptx_extract
```

4.4 Comprobacion de ofuscacion avanzada

```
grep -RaoE "([0-9a-fA-F]{2}){50,}" pptx_extract | head
```

```
grep -RaoE "([A-Za-z0-9+/]{40,}={0,2})" pptx_extract | head
```

```
grep -RaoE "(\\x[0-9a-fA-F]{2}){10,}" pptx_extract | head
```

```
grep -RaoE "(%[0-9a-fA-F]{2}){10,}" pptx_extract | head
```

- Si aparecen blobs largos, extraerlos y decodificarlos.
- Probar strings ASCII y UTF-16LE sobre cada artefacto.
- Usar floss o xorsearch si se sospecha ofuscacion simple.
- No interpretar ausencia de strings como ausencia de payload.

5. Extraccion de objetos y artefactos

5.1 Extraccion en .ppt/.pps OLE

```
oledump.py muestra.ppt
```

```
oledump.py --storages muestra.ppt
```

```
oledump.py -s NUMERO -d muestra.ppt > stream_NUMERO.bin
```

```
oledump.py -s NUMERO -v muestra.ppt > macro_NUMERO.vba
```

```
oleobj muestra.ppt -d extraccion-ppt
```

- Extraer streams sospechosos.
- Guardar macros desofuscadas si olevba las recupera.
- Extraer objetos OLE embebidos con oleobj.
- Conservar hashes de cada artefacto.

5.2 Extraccion en .pptx/.pptm OOXML

```
mkdir pptx_extract
```

Procedimiento de analisis estatico de PowerPoint maliciosos

```
unzip muestra.pptx -d pptx_extract

find pptx_extract/ppt/embeddings -type f -maxdepth 1 2>/dev/null

find pptx_extract/ppt/activeX -type f -maxdepth 1 2>/dev/null

find pptx_extract -name "vbaProject.bin" -print
```

- ppt/embeddings/ suele contener objetos OLE embebidos.
- ppt/activeX/ puede contener controles ActiveX y relaciones asociadas.
- ppt/vbaProject.bin contiene macros en documentos habilitados para macros.
- Los ficheros .rels indican como se conectan slides, objetos y recursos.

5.3 Validacion de artefactos extraidos

```
file artefacto

md5sum artefacto

sha256sum artefacto

xxd -l 64 artefacto
```

Cabecera	Formato probable	Siguiente paso
D0 CF 11 E0 A1 B1 1A E1	OLE/CFBF	oledump.py, oleid, oleobj, olevba
4D 5A	PE/EXE	pefile, capa, diec, strings, no ejecutar
50 4B 03 04	ZIP/OOXML/archivo comprimido	unzip, 7z, revisar contenido
Texto/XML	XML, script, relationship	grep, xmllint, revision manual
Sin cabecera clara	blob raw	strings, xxd, floss, emulacion si procede

6. Analisis de artefactos extraidos

6.1 Objetos OLE

```
oledump.py objeto.ole

oledump.py --storages objeto.ole

oleid objeto.ole

oleobj objeto.ole -d oleobj_out
```

- Si solo aparece Root Entry, puede ser OLE vacio, parcial o corrupto.
- Si aparecen streams, extraerlos y analizarlos individualmente.
- Si hay macros, analizarlas con olevba.
- Si hay objetos embebidos, volver a validar cada uno con file y xxd.

6.2 Macros VBA

```
olevba ppt/vbaProject.bin > vba.txt

olevba muestra.pptm > vba.txt
```

```
grep -Ei "Auto|Open|CreateObject|Shell|PowerShell|XMLHTTP|WinHttp|ADODB|URLDownload|Environ|Temp" vba.txt
```

- Identificar eventos de ejecucion automatica.
- Separar cadenas tecnicas de IOCs reales.
- Buscar comandos, URLs, rutas, dominios y payloads embebidos.
- Documentar desofuscacion paso a paso si hay concatenacion, Base64 o Chr.

6.3 Ejecutables, scripts y blobs raw

```
pecheck.py -l P artefacto
```

```
strings -a artefacto | grep -Ei "http|https|cmd|powershell|LoadLibrary|GetProcAddress|VirtualAlloc|URLDownload"
```

```
strings -a -el artefacto | grep -Ei "http|https|cmd|powershell|LoadLibrary|GetProcAddress|VirtualAlloc|URLDownload"
```

- Si aparece MZ/PE, analizar como ejecutable sin lanzarlo.
- Si hay shellcode probable, revisar contexto hexadecimal y desensamblar.
- Si hay scripts, abrirlos como texto y buscar ofuscacion.
- Si hay URLs externas, no acceder directamente desde el host.

7. Busqueda generica de cadenas en todos los artefactos

```
for f in $(find extraccion-ppt pptx_extract -type f 2>/dev/null); do
```

```
echo
```

```
echo "===== $f ====="
```

```
echo "[ASCII]"
```

```
strings -a -t x -n 4 "$f" | grep -Ei "http|https|www\.|\.\exe|\.\dll|\.\vbs|\.\js|cmd|powershell|wscript|mshta|shell|download|url|kernel|loadlibrary|getprocaddress|createprocess|winexec|shellexecute|appdata|temp|public|programdata|xmlhttp|adodb|winhttp" || true
```

```
echo "[UTF-16LE]"
```

```
strings -a -el -t x -n 4 "$f" | grep -Ei "http|https|www\.|\.\exe|\.\dll|\.\vbs|\.\js|cmd|powershell|wscript|mshta|shell|download|url|kernel|loadlibrary|getprocaddress|createprocess|winexec|shellexecute|appdata|temp|public|programdata|xmlhttp|adodb|winhttp" || true
```

```
done
```

- El objetivo es descubrir cadenas relevantes sin partir de IOCs conocidos.
- Registrar fichero, offset y codificacion de cada hallazgo.
- No concluir ejecucion solo por presencia de una cadena.
- Comparar resultados entre macros, embeddings, relationships y blobs raw.

8. Correlacion con la estructura original

La correlacion permite pasar de cadenas aisladas a una interpretacion estructural. En PowerPoint no siempre se correlaciona con un offset del documento original; muchas veces se correlaciona con rutas internas, streams OLE o relaciones OOXML.

Formato	Como correlacionar	Ejemplo
.ppt/.pps OLE	Stream o storage dentro del contenedor	Stream 12 contiene macro; storage Embeddings contiene OLE
.pptx/.pptm OOXML	Ruta interna del ZIP y fichero .rels	ppt/slides/_rels/slide1.xml.rels apunta a ppt/embeddings/oleObject1.bin
Objeto embebido	Hash, cabecera, offset interno y ruta de extraccion	oleObject1.bin contiene PE en offset 0x200

```
grep -Rai "oleObject\\|activeX\\|Target=\\|TargetMode=\"External\"\\|http\\|https" pptx_extract/_rels
pptx_extract/ppt 2>/dev/null
```

- Relacionar cada IOC con el artefacto donde aparece.
- En OOXML, revisar el fichero .rels que referencia cada objeto.
- En OLE, documentar numero de stream, nombre de storage y hash del stream extraido.
- Si una cadena aparece en un objeto embebido, explicar como se llega a ese objeto desde la presentacion.

9. Analisis de contexto hexadecimal y desensamblado

9.1 Contexto hexadecimal

```
xxd -g 1 -s OFFSET -l 0x200 artefacto.bin
```

- Buscar cadenas mezcladas con instrucciones.
- Buscar cabeceras desplazadas.
- Identificar patrones de shellcode, como call + datos inline.
- Revisar si hay padding, datos cifrados o compresion.

9.2 Desensamblado de blobs sospechosos

Ghidra / IDA Free / Cutter / radare2

Import as Raw Binary

Seleccionar arquitectura probable: x86, x64, VBA p-code si aplica

Ir al offset sospechoso

Forzar disassembly

Crear funcion manualmente

Marcar strings inline

- En blobs raw no hay entry point ni secciones.
- Puede ser necesario probar varios offsets.
- Buscar resolucion dinamica de APIs, parsing de PEB o tabla de exportaciones.
- Si aparecen APIs de red/descarga, formular hipotesis prudente y confirmar con emulacion.

10. Clasificación de hallazgos

Nivel	Ejemplos	Como redactarlo
Estructural	OLE, ZIP/OOXML, embeddings, activeX, vbaProject.bin, .rels	La muestra contiene objetos embebidos o relaciones externas
Tecnico	LoadLibrary, GetProcAddress, CreateObject, XMLHTTP, ADODB.Stream, shellcode probable	Compatible con resolucion dinamica, descarga o ejecucion
IOC	URL, dominio, IP, hash, ruta, nombre de fichero	Indicador concreto observable o correlacionable

- No todo hallazgo tecnico es un IOC.
- kernel32 o LoadLibrary son cadenas tecnicas, no IOCs por si solas.
- Una URL, dominio, ruta o hash si puede documentarse como IOC.
- Las conclusiones deben indicar nivel de certeza.

11. Conclusion estatica prudente

La conclusion del analisis estatico debe evitar afirmar ejecucion real si solo se han observado cadenas, macros u objetos embebidos.

Ejemplo:

El analisis estatico identifica una presentacion PowerPoint con objetos embebidos y artefactos sospechosos. En los artefactos extraidos se observan cadenas asociadas a ejecucion, descarga o resolucion dinamica de APIs, junto con posibles URLs o rutas de escritura.

Estos elementos son compatibles con un downloader o dropper embebido. No obstante, la ejecucion real de las APIs identificadas, la descarga del payload o la creacion de ficheros deberan confirmarse mediante emulacion o analisis dinamico en entorno controlado.

12. Cuando pasar a emulacion o analisis dinamico

- Cuando haya shellcode o blob raw con APIs relevantes.
- Cuando haya macros que construyen comandos o descargan payloads.
- Cuando haya objetos embebidos que no se puedan interpretar completamente en estatico.
- Cuando se necesite confirmar si una URL, ruta o API se usa realmente durante ejecucion.

```
speakeasy.exe -t blob.raw --raw -a x86 -o salida.json
```

```
speakeasy.exe -t blob.raw --raw -a x86 --raw_offset 0x100 -o salida_100.json
```

```
scdbg.exe -f blob.bin -s 2000000
```

Para PowerPoint con macros, el analisis dinamico debe realizarse en una maquina virtual aislada, monitorizando PowerPoint, procesos hijos, sistema de ficheros, registro y red. Para blobs raw, Speakeasy, scdbg, Unicorn o Qiling pueden ser mas adecuados que x32dbg, salvo que se prepare un loader de shellcode.

Procedimiento de analisis estatico de PowerPoint maliciosos

13. Checklist rapido

- Calcular hashes y confirmar tipo real del fichero.
- Distinguir .ppt/.pps OLE de .pptx/.pptm OOXML.
- En .ppt/.pps: oleid, oledump.py, olevba, oleobj.
- En .pptx/.pptm: unzip, revisar [Content_Types].xml, .rels, embeddings, activeX y vbaProject.bin.
- Buscar relaciones externas y TargetMode="External".
- Extraer objetos embebidos y validar cada artefacto con file, xxd y hashes.
- Buscar macros, scripts, PE embebidos, blobs raw y shellcode probable.
- Realizar strings ASCII y UTF-16LE en todos los artefactos.
- Correlacionar cada hallazgo con stream OLE, ruta OOXML o fichero .rels.
- Analizar contexto hexadecimal de cadenas relevantes.
- Usar desensamblador si hay shellcode o blob raw.
- Separar hallazgos estructurales, hallazgos tecnicos e IOCs.
- Redactar conclusion estatica prudente.
- Pasar a emulacion/dinamico solo cuando sea necesario confirmar comportamiento.